Chapitre 3

Architectures des applications réparties

Définition

L'architecture d'une application répartie désigne la manière dont les différents composants logiciels d'une application sont organisés, répartis sur plusieurs machines et coordonnent leurs interactions afin de fournir un service global.

C'est la structure qui décrit :

- ✓ où sont placés les composants ?
- ✓ comment ils communiquent ?
- ✓ comment ils coopèrent pour réaliser une fonctionnalité?

➡ L'architecture sert de plan d'organisation d'une application exécutée sur plusieurs ordinateurs connectés par un réseau.

Evolution des architectures

Architecture client-serveur : Répartition des rôles entre un client et un serveur.

Architecture orientée services (SOA) : pour faciliter l'intégration entre systèmes via les services Web.

Années 1960 → Années 1980 → Années 1990 → Années 2000 → Années 2010+

Architecture centralisée : Un seul ordinateur puissant partagé par plusieurs utilisateurs.

Architecture multi-couches : apparition des serveurs d'applications et des middleware comme CORBA.

Les architectures micro-services, évenementielles et serverless.

Qualités architecturales

L'architecture d'une application répartie cherche à assurer plusieurs qualités essentielles :

1. Scalabilité

- La scalabilité est la capacité du système à augmenter efficacement ses ressources pour supporter une charge plus élevée sans dégradation notable des performances.

2. Performance

- La performance désigne la rapidité avec laquelle le système traite les requêtes et répond aux utilisateurs tout en utilisant ses ressources de manière optimale.

3. Fiabilité

- La fiabilité est la capacité du système à fonctionner correctement sur une longue durée sans erreurs ni interruptions.

Qualités architecturales

L'architecture d'une application répartie cherche à assurer plusieurs qualités essentielles :

4. Tolérance aux pannes

- La tolérance aux pannes correspond à l'aptitude du système à continuer de fonctionner malgré la défaillance de l'un de ses composants.

5. Disponibilité

- La disponibilité est la capacité du système à rester opérationnel et accessible aux utilisateurs quasiment en permanence.

6. Sécurité

- La sécurité assure la protection des données et des services contre les accès non autorisés, les attaques et les manipulations malveillantes.

Qualités architecturales

L'architecture d'une application répartie cherche à assurer plusieurs qualités essentielles :

7. Maintenabilité

- La maintenabilité est la facilité avec laquelle le système peut être corrigé, modifié ou amélioré sans perturber son fonctionnement global.

8. Modularité

- La modularité désigne la capacité du système à être composé de modules indépendants, facilitant le développement, la compréhension et le remplacement des composants.

9. Faible couplage

- Le faible couplage signifie que les composants du système interagissent avec un minimum de dépendances, permettant une évolution indépendante des modules.

Qualités architecturales

L'architecture d'une application répartie cherche à assurer plusieurs qualités essentielles :

10. Interopérabilité

- L'interopérabilité est la capacité du système à communiquer et à fonctionner avec d'autres systèmes ou technologies, quels que soient leurs langages ou plateformes.

11. Transparence

- La transparence vise à masquer la complexité et la distribution du système, afin que l'utilisateur ou le programmeur perçoive un fonctionnement homogène et simple.

12. Compatibilité

- La compatibilité désigne la capacité du système à fonctionner correctement avec d'autres logiciels, protocoles ou versions, afin de garantir une intégration harmonieuse.

Les grandes familles d'architectures réparties

Les architectures classiques

- L'architecture client/serveur
- L'architecture en couches
- L'architecture Pair-à-pair

Les architectures modernes

- L'architecture orientée Service (SOA)
- L'architecture micro-services
- L'architecture serverless

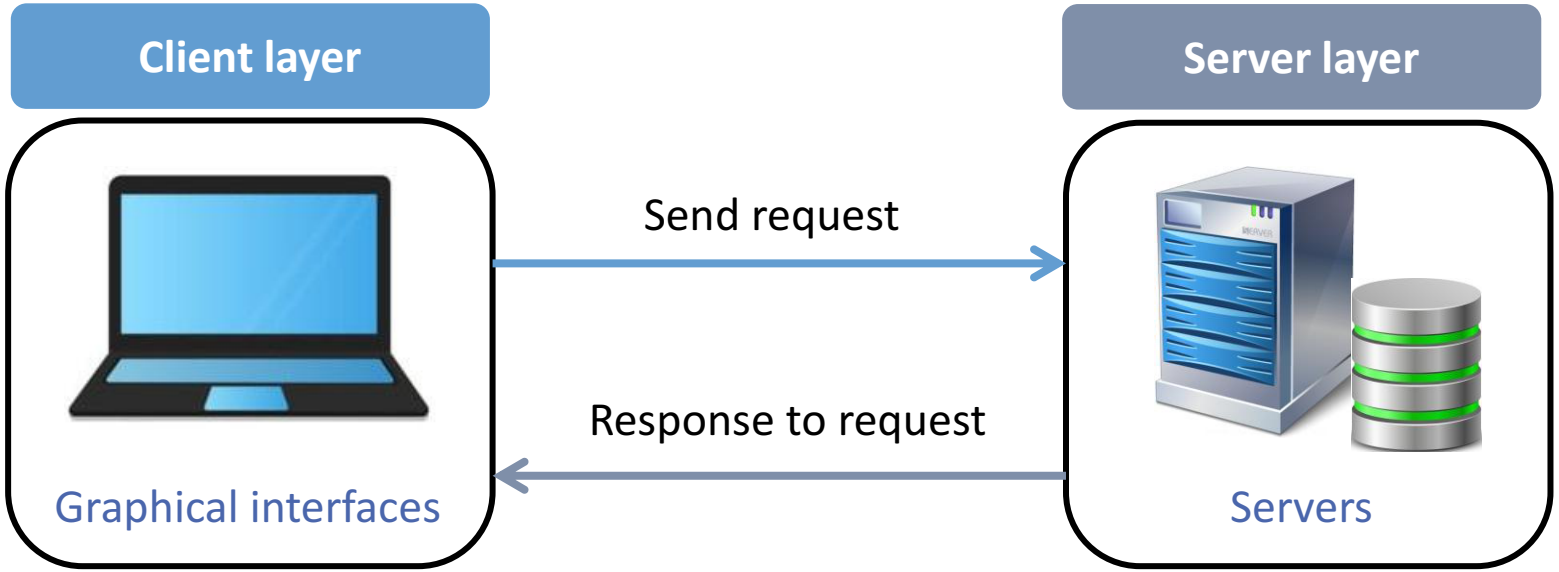
Architecture classique : Client/Serveur

L'architecture client–serveur est un modèle où l'application est divisée en deux parties distinctes.

Le client envoie des requêtes au serveur, qui traite la demande et renvoie une réponse.

➡ Cette architecture repose sur le principe d'accès à des ressources centralisées via un réseau.

Exemple : une application desktop qui interroge une base de données sur un serveur.



Architecture classique : en couches

L'architecture en couches est un modèle conceptuel qui organise une application en plusieurs niveaux logiques, chacun rempli une fonction spécifique.

Chaque couche possède une responsabilité propre (présentation, logique métier, accès aux données, etc.) et ne communique qu'avec les couches voisines.

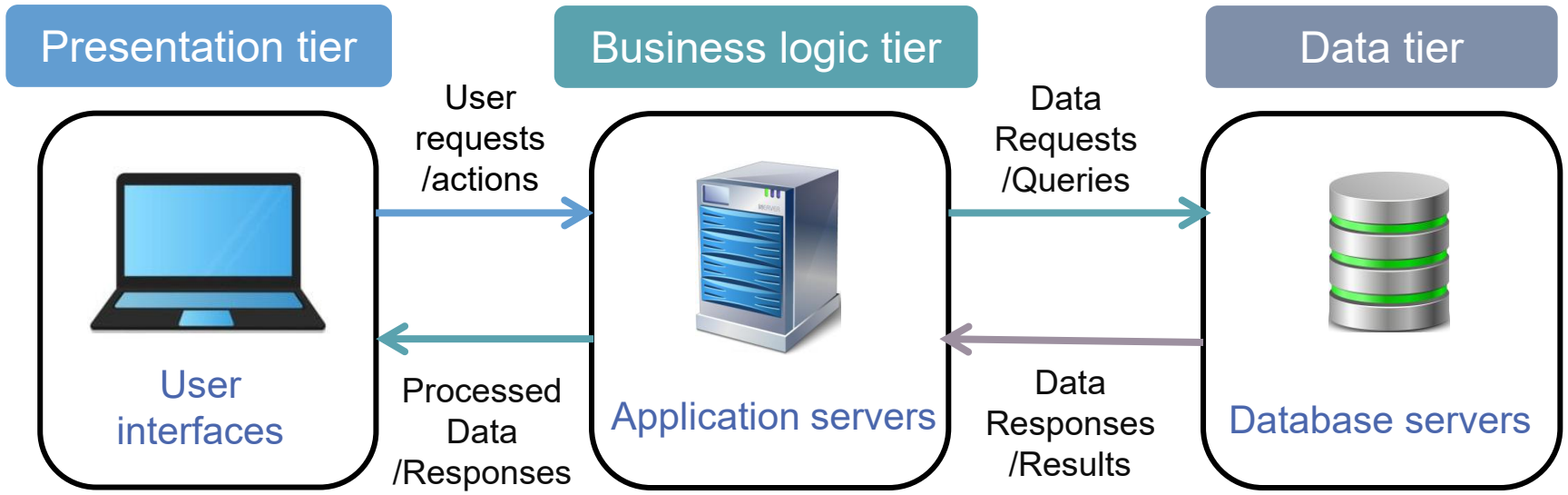
Objectifs : améliorer la modularité, faciliter la maintenance, encourager la réutilisabilité et clarifier la séparation des préoccupations.

Dans les applications réparties, l'architecture en couches peut être déployée physiquement sur plusieurs machines. Parmi les architectures qui implémentent ce modèle conceptuel, on trouve notamment :

- ✓ L'architecture 3-tiers, qui sépare physiquement l'interface utilisateur, la logique applicative et les données.
- ✓ Les architectures n-tiers basées sur le Web, qui rajoutent des tiers supplémentaires tels que le serveur Web.

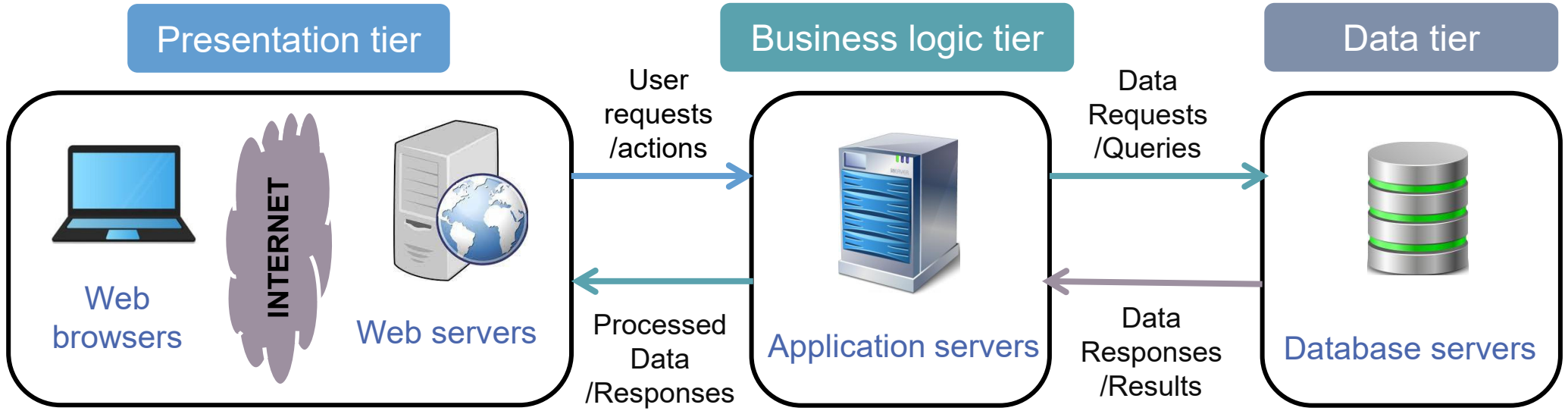
➡ Ces architectures sont donc des déclinaisons physiques et distribuées de l'architecture en couches.

Implémentation physique de l'architecture en couches



Architecture 3-tiers

Architecture n-tiers

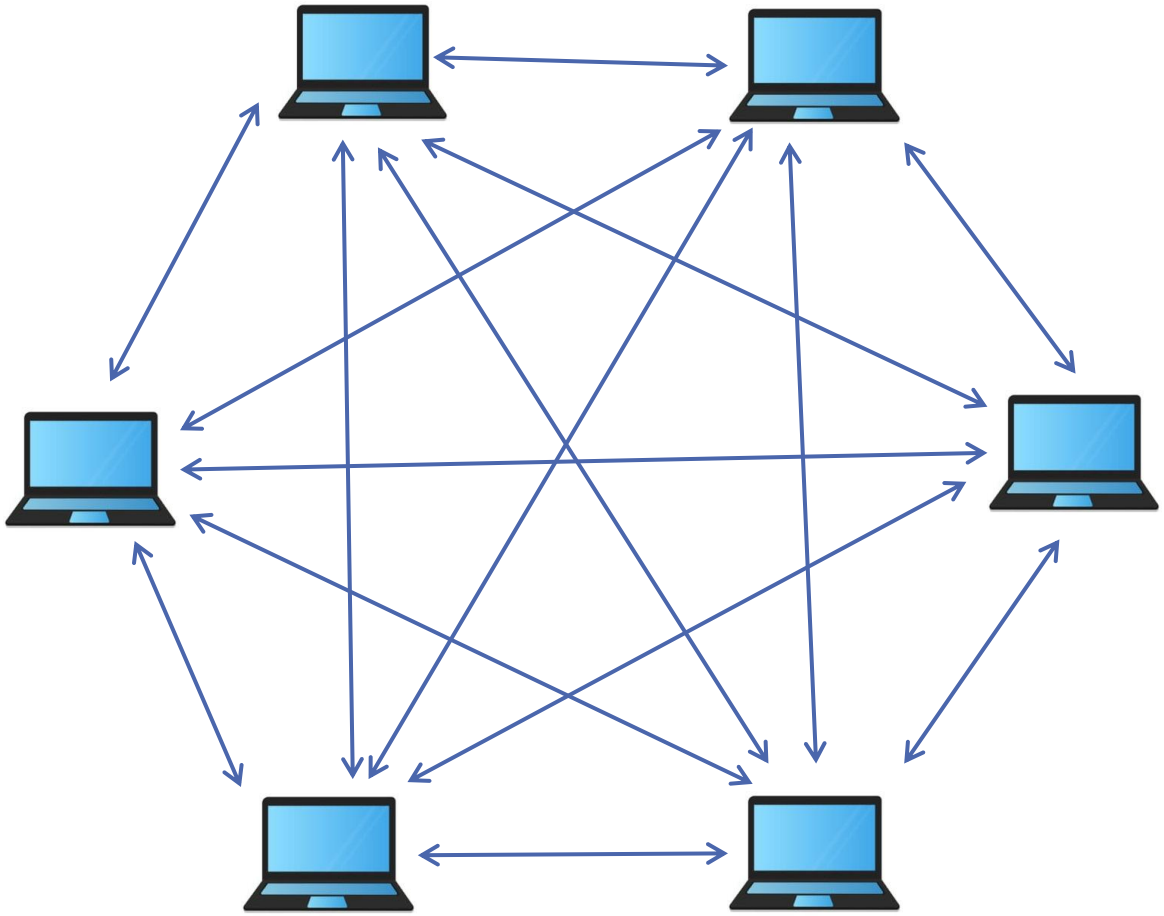


Architecture classique : P2P

L'architecture Pair-à-Pair (P2P) est un modèle d'organisation distribué dans lequel chaque nœud du réseau, appelé pair, joue simultanément le rôle de client et de serveur.

Contrairement à l'architecture client-serveur, il n'existe pas de serveur central : les pairs communiquent directement entre eux, partagent des ressources (fichiers, puissance de calcul, stockage, …) et collaborent pour assurer le fonctionnement global du système.

Le réseau P2P est décentralisé, autonome, scalable et robuste, car le système continue de fonctionner même si plusieurs nœuds tombent en panne.



Comparaison des architectures classiques

Critère	Architecture Client/Serveur	Architecture en Couches	Architecture Pair-à-Pair (P2P)
Organisation	1 serveur central, plusieurs clients.	Plusieurs couches logiques dépendant les unes des autres (2, 3, n couches).	Réseau maillé de pairs interconnectés sans serveur central.
Communication	Client → Serveur.	Entre couches voisines (présentation ↔ métier ↔ données).	Directement entre pairs (Peer ↔ Peer).
Centralisation	Forte centralisation.	Dépend du déploiement (souvent centralisé dans les tiers).	Décentralisation complète ou partielle.
Scalabilité	Limitée par la capacité du serveur.	Bonne scalabilité si les tiers sont distribués.	Très élevée : chaque nouveau pair ajoute des ressources.

Comparaison des architectures classiques

Critère	Architecture Client/Serveur	Architecture en Couches	Architecture Pair-à-Pair (P2P)
Tolérance aux pannes	Faible : si le serveur tombe, tout s'arrête.	Moyenne : dépend des tiers critiques.	Très élevée : le réseau continue même si plusieurs pairs tombent.
Complexité de mise en œuvre	Faible à moyenne.	Moyenne (plus de couches à gérer).	Plus complexe.
Avantages principaux	- Simplicité, - contrôle centralisé, maintenance facilitée.	- Modulaire, - maintenable, - réutilisable.	- Scalabilité, - robustesse, - absence de point unique de défaillance.
Inconvénients principaux	- Point unique de défaillance, - surcharge du serveur.	- Complexité accrue, - risque de surcouches inutiles.	- Sécurité complexe, - performances variables, - cohérence difficile.

Architecture moderne : orientée-service

★ Motivations :

- Difficulté d'intégrer des systèmes hétérogènes (Java, .NET, C++, …).
- Complexité des intégrations « point à point » dans les architectures classiques.
- Manque d'agilité pour adapter rapidement les systèmes aux nouveaux besoins.
- Besoin de réutiliser les fonctionnalités métier au lieu de les dupliquer (authentification, …).
- Nécessité de standardiser la communication entre applications.

★ Définition de l'architecture SOA :

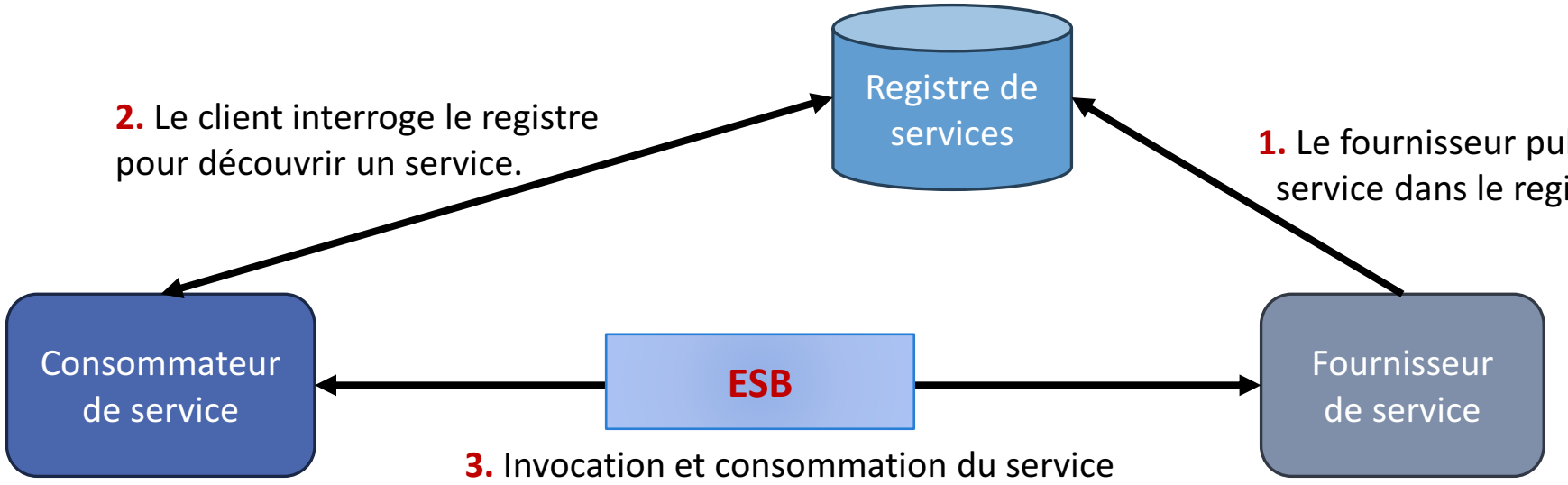
L'architecture orientée services est un style d'architecture distribuée dans lequel les fonctionnalités d'un système sont organisées sous forme de services indépendants, accessibles à distance via des interfaces standardisées (REST, SOAP, …).

Un service est une unité fonctionnelle autonome, faiblement couplée, qui réalise une tâche métier bien précise et peut être utilisé par plusieurs applications.

Architecture moderne : orientée-service

★ Les composants de l'architecture SOA :

- ✓ Les Consommateurs de Services : Applications ou systèmes qui invoquent les services.
- ✓ Les Fournisseurs de Services : Systèmes qui implémentent et publient les services.
- ✓ L'Annuaire ou Registre de Services : Référencie les services disponibles et permet leur découverte par les consommateurs.
- ✓ Le Mécanisme d'Intégration (Enterprise Service Bus ESB) : Assure le routage, la transformation et la médiation entre services.



Architecture moderne : microservices

L'architecture microservices peut être considérée comme une évolution moderne et plus fine de la SOA.

★ Motivation :

- Alléger la complexité des applications SOA (souvent trop centralisées autour de l'ESB).
- Accélérer le développement et les mises à jour grâce à des services plus petits et indépendants.
- Permettre une évolutivité fine : ne scaler qu'un microservice au lieu de l'application entière.
- Adopter l'indépendance technologique (chaque microservice peut utiliser son langage, sa base, son framework).
- Améliorer la résilience : une panne d'un microservice ne fait pas tomber toute l'application.

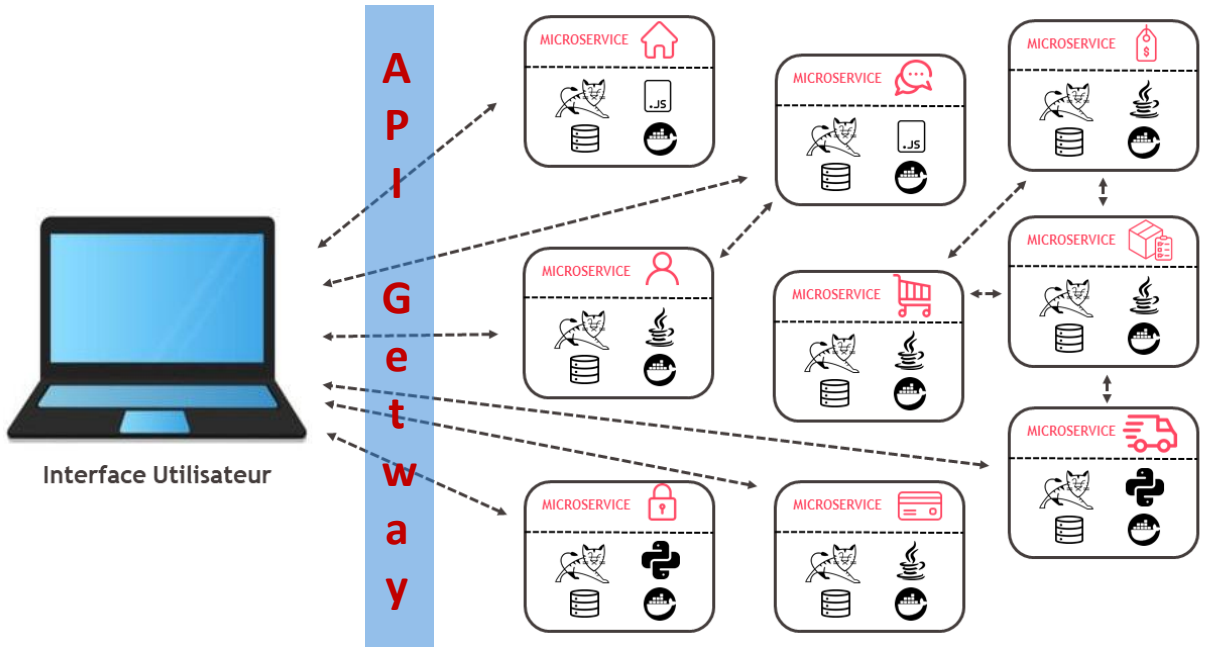
★ Définition :

L'architecture microservices est un style d'architecture où une application est décomposée en un ensemble de petits services indépendants et autonomes.
Chaque microservice exécute son propre processus, possède son propre espace de déploiement et de données, expose sa fonctionnalité via des API légères (REST, gRPC, …), se concentre sur une capacité métier unique et peut être développé, testé, déployé et scalé indépendamment des autres.

Architecture moderne : microservices

★ Les composants de l'architecture microservices :

- ✓ API Gateway : Point d'entrée unique pour les clients ; elle reçoit les requêtes et les redirige vers les microservices appropriés.
- ✓ Conteneurs (ex: Docker) : Environnement léger dans lequel chaque microservice est emballé (le service + son environnement d'exécution) pour être facilement déployé sur n'importe quelle machine.
- ✓ Orchestrateur (ex : Kubernetes) : Plateforme qui gère automatiquement les conteneurs : déploiement, redémarrage, scaling, réseau interne, etc.
- ✓ Système de communication : Mécanisme utilisé par les microservices pour échanger (API REST ou gRPC en mode synchrone, ou messages/événements en mode asynchrone).



Architecture moderne : évènementielle

L'architecture orientée événements (Event-Driven Architecture, EDA) est un modèle de conception logicielle où les composants réagissent à des événements, ou à des changements d'état, au lieu de dépendre de requêtes directes. Les événements sont publiés par des "producteurs d'événements" et consommés par des "consommateurs d'événements" via un canal d'événements ou un broker, ce qui permet une communication découplée et asynchrone.

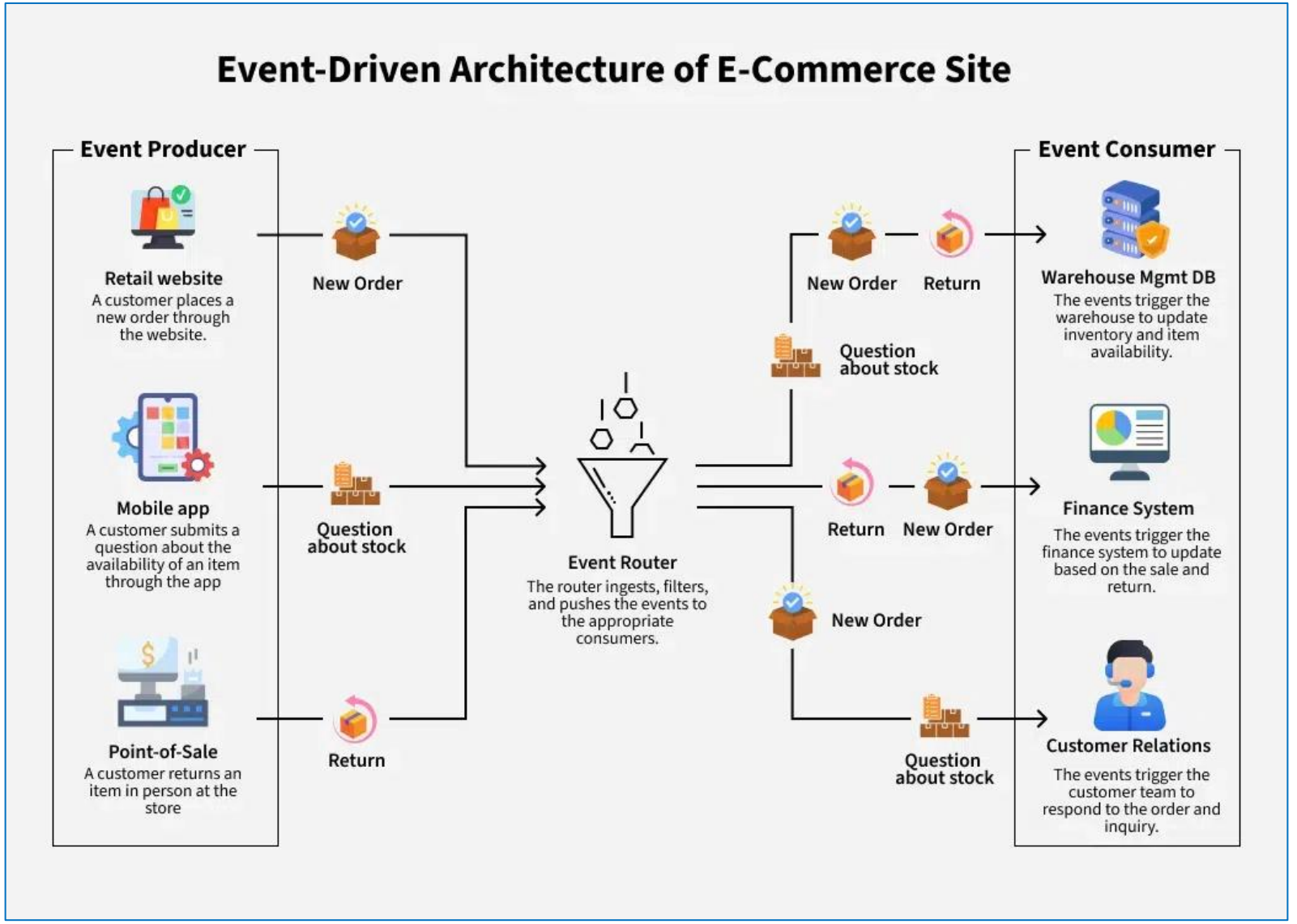
★ Motivation :

- Réactivité : permettre aux systèmes de réagir immédiatement à des événements.
- Découplage : séparer fortement les composants.
- Scalabilité : gérer facilement de grands volumes d'événements grâce à un traitement asynchrone.

★ Les composants de l'architecture évènementielle :

- ✓ Producteurs d'événements : Ce sont les composants qui génèrent des événements.
- ✓ Consommateurs d'événements : Ce sont les composants qui écoutent des événements spécifiques et effectuent une action lorsqu'un événement survient.
- ✓ Canaux d'événements : Ce sont les intermédiaires (comme un broker d'événements) qui reçoivent les événements des producteurs et les acheminent vers les consommateurs appropriés.

Architecture moderne : évènementielle



Architecture moderne : serverless

★ Définition :

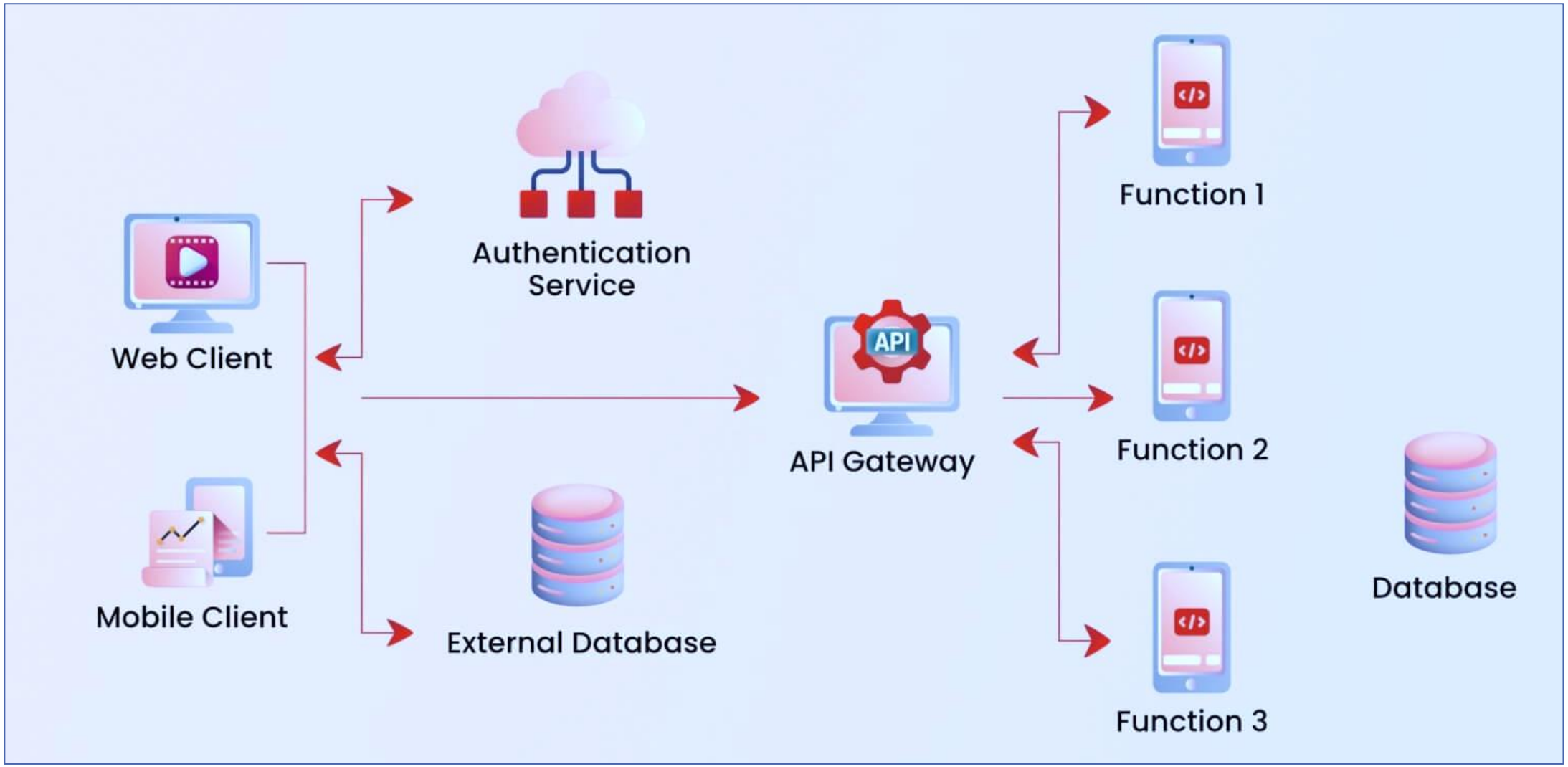
L'architecture serverless est un modèle de développement où l'application repose sur des services cloud entièrement gérés par les fournisseurs cloud, dans lesquels le développeur ne s'occupe plus de la gestion des serveurs, ni de leur configuration, ni de leur mise à l'échelle.
L'exécution du code se fait à la demande, sous forme de petites fonctions (Functions-as-a-Service), déclenchées par des événements (requête HTTP, message, événement d'un autre service, etc.).

→ Les fonctions sont déployées et exécutées sur une infrastructure cloud distribuée.
→ L'application est composée de services découplés, chacun pouvant être déployé et exécuté indépendamment.

★ Motivation :

- Réduire la complexité opérationnelle et accélérer le développement
- S'adapter automatiquement à la charge
- Payer uniquement à l'usage

Architecture moderne : serverless



Comparaison des architectures modernes

Critère	SOA	Microservices	Architecture Événementielle	Serverless
Principe	Services réutilisables communiquant via un bus (ESB).	Petits services autonomes, faiblement couplés.	Système basé sur l'échange d'événements pour déclencher des actions.	Exécution de fonctions à la demande, sans gérer de serveurs.
Granularité	Services grands et souvent complexes.	Services petits et spécialisés.	Événements + traitements découpés en réactions.	Fonctions très petites et unitaires.
Couplage	Couplage modéré (souvent via un ESB central).	Très faible couplage.	Très faible couplage (communication asynchrone).	Très faible couplage (à l'événement).
Communication	Souvent synchrones (SOAP).	API légères (REST/JSON).	Majoritairement asynchrone (files, messages).	Déclenchement par événements (HTTP, stockage, file…).
Scalabilité	Limitée par l'ESB.	Très élevée, indépendante (par service).	Très élevée selon le flux d'événements.	Scalabilité automatique gérée par le cloud.

Comparaison des architectures modernes

Critère	SOA	Microservices	Architecture Événementielle	Serverless
Gestion d'infrastructure	L'entreprise gère les serveurs et l'ESB.	L'entreprise gère les conteneurs/machines.	L'entreprise gère les composants événementiels.	Infrastructure 100 % gérée par le cloud.
Cas d'usage typiques	Intégration de grands systèmes d'entreprise.	Applications web/Cloud complexes et modulaires.	IoT, systèmes réactifs, streaming.	Traitements ponctuels, automatisations.
Avantages	Réutilisation, standardisation.	Rapidité de déploiement, flexibilité.	Réactivité, découplage fort, performance.	Coût réduit, aucune gestion serveur, auto-scaling.
Inconvénients	ESB peut devenir un goulot.	Gestion complexe du déploiement (DevOps).	Difficulté à gérer l'ordre, la cohérence.	Limitations de durée, dépendance au cloud.